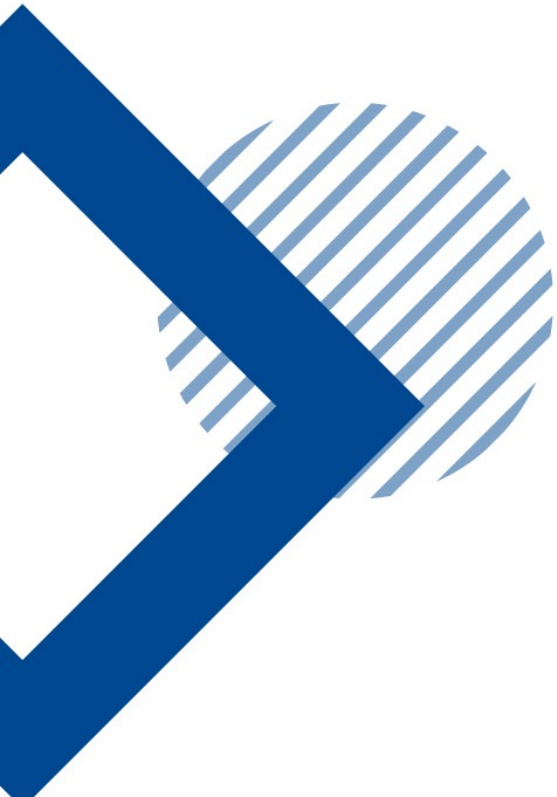
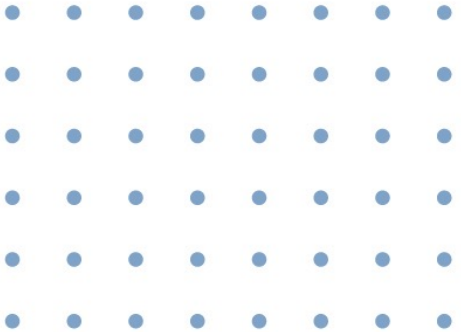
JavaScript, AJAX, REST API

Интернет-программирование







2022

JavaScript

JavaScript это:

- Интерпретируемый язык. Его интерпретатор обычно встроен в браузер.
- Основное назначение – определять «динамическое» поведение страниц при загрузке (формирование страницы в браузере на стороне клиента) и при работе пользователя со страницей (UI элементы).
- Текст на JavaScript может быть вложен в HTML-страницу непосредственно или находиться в отдельном файле (как CSS).
- Похож на языки Java и C# синтаксически, но сильно отличается от них по внутреннему содержанию.

Характеристика JavaScript

Язык объектно-ориентированного программирования. Объекты в языке имеют «тип», «атрибуты» и «методы»

"John,Jane,Paul,Michael".split(",").length

Переменные не имеют заранее заданного типа, то есть в разные моменты времени могут содержать значения разных типов

var number = 25; number = (number < 0); number = "25";

Типы объектов могут быть: number, string, function, object, undefined. Оператор typeof позволяет «вычислить» тип объекта

typeof 25 == "number"   typeof null == "object”

Функции можно передавать как аргументы

form.addEventListener('submit', function(event) { return false; });

Включение JavaScript в HTML-страницу

Внутри страницы.

```
<script>
        alert(‘Привет, мир!’);
</script>
```

Внутри тега.

```
<button onclick=”alert(‘Привет, мир!’);”>Нажми на меня!</button>
```

Отдельно от разметки.

```
<button id=”hello”>Нажми на меня!</button>
<script>
   var button = document.getElementById(”hello");
   button.addEventListener(‘click’, function() {
      alert(‘Привет, мир!’);
   };
);
</script>
```

В отдельном файле.

```
<script src="js/banner.js"></script>
```

Основные встроенные типы

В JS есть набор встроенных «классов», порождающих «объекты», различающиеся набором атрибутов и методов.

Программисты могут динамически изменять поведение этих «классов» и создавать свои собственные.

Каждый «класс» является объектом, у которого есть «прототип», определяющий набор атрибутов и методов у всех вновь создаваемых объектов этого класса.

Типы, встроенные в язык, это:

- Number : 64-х-разрядные числа с плавающей точкой.
- String : строки с символами в формате Unicode.
- Array : массивы с переменными границами.
- Function : Функции. Каждая функция, кроме того, может служить конструктором объекта.
- Boolean, Date, Math, RegExp : логические значения, даты и т. д.

Приведение типов



Объекты, встроенные в браузеры

- window : представляет «глобальный контекст» и позволяет работать с атрибутами и методами окна.
- document : загруженная страница со своей структурой элементов.
- navigator : объект, представляющий браузер и его свойства.
- location : характеристики текущего URL (порт, хост и т.п.).
- объекты, представляющие элементы различных типов в HTML-странице, такие как <body>, <link>, <img> и т.п.
- события (events), возникающие от действий пользователя, например, нажатие кнопки мыши (click), загрузка новой страницы (load) и т.д.

События и реакции на них

Имеется большое количество событий, которые можно разделить на следующие классы:

- события от мыши (click, dblclick, mousedown,...);
- события от клавиатуры (keypress, keydown,...);
- события от элементов ввода (focus, submit, select,...);
- события страницы (load, unload, error,...).

Один из способов программирования состоит в определении реакции на события непосредственно в описании элемента, например:

<button onclick=”alert(‘Привет, мир!’);”>Нажми на меня!</button>

Недостаток этого способа: javascript-текст опять смешивается с содержанием страницы.

Работа с таймером

Можно создать таймер и определить реакцию на событие от таймера.

var timer = setTimeout(func, timeinterval);
func – это функция или строка с кодом;
timeinterval – время в миллисекундах.
Таймер срабатывает один раз и запускает функцию.

function launchTimer() { setTimeout("alert('Зенит – чемпион!');", 2000); }

Теперь можно запустить этот таймер, например, по событию click:
<body> Нажми сюда! </body>

Пока событие еще не случилось, таймер можно остановить:
var timer = setTimeout(func, timeinterval);
clearTimeout(timer);

Работа с интервальным таймером

Таймер может срабатывать многократно через равные промежутки времени. Такой таймер создается с помощью функции setInterval и останавливается с помощью функции clearInterval.

```
var timer = setInterval(func, timeinterval);

function launchInterval() {
        timer = setInterval("alert('Зенит – чемпион!');", 2000);
}

function stopTimer() {
        if (timer) clearInterval(timer);
        timer = null;
}

<body>
        <p>Нажми <span onclick="launchInterval();">сюда,</span> чтобы запустить.</p>
        <p>Нажми <span onclick="stopTimer();">сюда,</span> чтобы остановить.</p>
</body>
```

Объектная модель браузера

Объектная модель браузера – браузероспецифичная часть языка, являющаяся прослойкой между JS и объектной моделью документа.

Основное предназначение объектной модели браузера – управление окнами браузера и обеспечение их взаимодействия.

Каждое из окон браузера представляется объектом window, центральным объектом BOM.

Функции BOM

- Управление фреймами.
- Поддержка задержки в исполнении кода и зацикливания с задержкой.
- Системные диалоги.
- Управление адресом открытой страницы.
- Управление информацией о браузере.
- Управление информацией о параметрах монитора.
- Ограниченное управление историей просмотра страниц.
- Поддержка работы с HTTP cookie.

Объектная модель документа

DOM (от англ. Document Object Model – «объектная модель документа») – это не зависящий от платформы и языка программный интерфейс, позволяющий программам и скриптам получить доступ к содержимому HTML, XHTML и XML-документов, а также изменять содержимое, структуру и оформление таких документов.

Объектная модель документа – интерфейс программирования приложений для HTML и XML-документов.

Согласно DOM документу можно поставить в соответствие дерево объектов, обладающих рядом свойств, которые позволяют производить с ним различные манипуляции:

- получение узлов,
- изменение узлов,
- изменение связей между узлами,
- удаление узлов.

Иерархия объектов в HTML DOM

Модель DOM не накладывает ограничений на структуру документа.

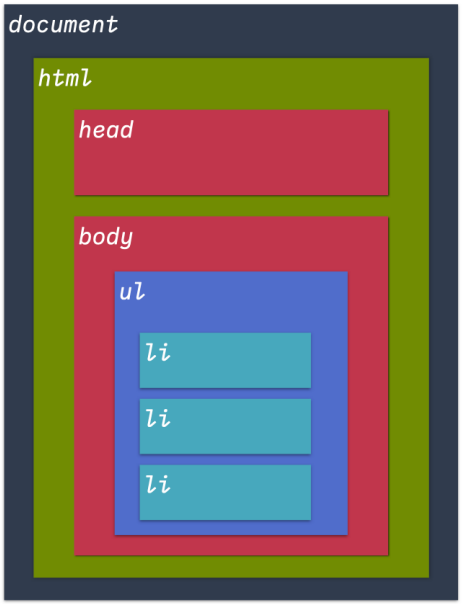
Любой документ известной структуры с помощью DOM может быть представлен в виде дерева узлов, каждый узел которого представляет собой элемент, атрибут, текстовый, графический или любой другой объект.

Узлы связаны между собой отношениями родительский-дочерний.

```
<html lang="en">
  <head>…</head>
  <body>
    …
    <ul>
      <li>🐓 dance</li>
      <li>🎉 cheer</li>
      <li>🌮 eat</li>
    </ul>
  </body>
</html>
```

Pick your party style!

- 🐓 dance
- 🎉 cheer
- 🌮 eat



@MarieChatfield

AJAX

AJAX, Ajax (Asynchronous Javascript and XML — «асинхронный JavaScript и XML») — подход к построению интерактивных пользовательских интерфейсов веб-приложений, заключающийся в «фоновом» обмене данными браузера с веб-сервером.

В результате при обновлении данных веб-страница не перезагружается полностью, и веб-приложения становятся быстрее и удобнее.

Синхронный запрос блокирует работу клиентского кода до момента возврата ответа от сервера.

Асинхронный код не блокирует работу клиентского кода, но требует определенной работы с запросом к серверу через функции обратного вызова (callback).

XMLHttpRequest

Fetch API

REST API – 1

REST (от англ. Representational State Transfer — «передача репрезентативного состояния» или «передача „самоописываемого“ состояния») — архитектурный стиль взаимодействия компонентов распределённого приложения в сети.

При использовании REST API все взаимодействие между клиентом и сервером происходит через протокол HTTP.

Протокол HTTP позволяет обратиться к URL на сервере с использованием глаголов:

- GET – получение данных;
- POST –создание данных;
- PUT/PATCH – обновление/изменение данных;
- DELETE – удаление данных.

REST API – 2

При использовании REST API с некоторым объектом (сущностью) связан некоторый адрес (URL).
Базовый REST API реализует CRUD-методы:

- Create (HTTP Post) – данные передаются через тело запроса;
- Read (HTTP Get) – данные передаются через фрагмент пути (path id) URL;
- Update (HTTP Put) – данные передаются через тело запроса и фрагмент пути URL;
- Delete (HTTP Delete) - данные передаются через фрагмент пути URL.

Остальные методы добавляются к базовому URL в виде дополнительного пути (path), например, http://localhost/api/orders/1/close

Спасибо за внимание!

Литература:

1. https://developer.mozilla.org/ru/docs/Web/JavaScript — MDN web docs: JavaScript.
2. https://developer.mozilla.org/ru/docs/Web/API/XMLHttpRequest - XMLHttpRequest
3. https://developer.mozilla.org/ru/docs/Web/API/Fetch_API - Fetch API
4. https://www.npmjs.com/package/json-server – JSON server
5. https://medium.com/codingthesmartway-com-blog/create-a-rest-api-with-json-server-36da8680136d - Create A REST API With JSON Server



2022